

Terraform

Modules





Introduction to Terraform Modules

Terraform modules are containers for multiple resources that are used together. They allow you to group resources together and reuse this group later, possibly many times. Modules can be thought of as "functions" in infrastructure as code - they encapsulate complexity, promote reusability, and make your configuration more maintainable.

Types of Modules

1. **Root Module:** Every Terraform configuration has at least one module, called the root module, which consists of the .tf files in your working directory.
2. **Child Modules:** These are modules called by other modules (including the root module).
3. **Published Modules:** Modules that are published in Terraform Registry or other repositories.

Module Structure

A typical module has this structure:

module-name/

- ├─ main.tf # Primary resources
- ├─ variables.tf # Input variables
- ├─ outputs.tf # Output values
- ├─ README.md # Documentation
- └─ examples/ # Example usage

Project Structure





```
terraform-modules-demo/
├── main.tf                # Root module
├── variables.tf           # Root variables
├── outputs.tf            # Root outputs
├── modules/
│   ├── networking/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── compute/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── database/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
└── examples/
    ├── dev/
    │   ├── main.tf
    │   └── terraform.tfvars
    └── prod/
        ├── main.tf
        └── terraform.tfvars
```

1. Root Module Configuration

main.tf





```
terraform {
  required_version = ">= 1.0.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

module "networking" {
  source          = "../modules/networking"
  vpc_cidr        = var.vpc_cidr
  public_subnets = var.public_subnets
  private_subnets = var.private_subnets
  environment     = var.environment
}

module "compute" {
  source          = "../modules/compute"
  vpc_id          = module.networking.vpc_id
  public_subnet_ids = module.networking.public_subnet_ids
  private_subnet_ids = module.networking.private_subnet_ids
  instance_type    = var.instance_type
  key_name         = var.key_name
  environment      = var.environment
}

module "database" {
  source          = "../modules/database"
  vpc_id          = module.networking.vpc_id
  private_subnet_ids = module.networking.private_subnet_ids
  db_name         = var.db_name
  db_username     = var.db_username
  db_password     = var.db_password
  environment     = var.environment
}
```

variables.tf





```
variable "aws_region" {
    description = "AWS region to deploy resources"
    type        = string
    default     = "us-east-1"
}

variable "environment" {
    description = "Environment name (dev, staging, prod)"
    type        = string
}

variable "vpc_cidr" {
    description = "CIDR block for VPC"
    type        = string
    default     = "10.0.0.0/16"
}

variable "public_subnets" {
    description = "List of public subnet CIDR blocks"
    type        = list(string)
    default     = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "private_subnets" {
    description = "List of private subnet CIDR blocks"
    type        = list(string)
    default     = ["10.0.3.0/24", "10.0.4.0/24"]
}

variable "instance_type" {
    description = "EC2 instance type"
    type        = string
    default     = "t2.micro"
}

variable "key_name" {
    description = "Name of the EC2 key pair"
    type        = string
}
```





```
variable "db_name" {  
  description = "Database name"  
  type        = string  
}  
  
variable "db_username" {  
  description = "Database username"  
  type        = string  
}  
  
variable "db_password" {  
  description = "Database password"  
  type        = string  
  sensitive   = true  
}
```

outputs.tf





```
output "vpc_id" {
  description = "ID of the VPC"
  value       = module.networking.vpc_id
}

output "web_server_public_ips" {
  description = "Public IP addresses of web servers"
  value       = module.compute.web_server_public_ips
}

output "database_endpoint" {
  description = "Endpoint of the RDS database"
  value       = module.database.database_endpoint
}

output "alb_dns_name" {
  description = "DNS name of the Application Load Balancer"
  value       = module.compute.alb_dns_name
}
```

2. Networking Module

modules/networking/main.tf





```
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = {
    Name      = "${var.environment}-vpc"
    Environment = var.environment
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags = {
    Name      = "${var.environment}-igw"
    Environment = var.environment
  }
}

resource "aws_subnet" "public" {
  count          = length(var.public_subnets)
  vpc_id         = aws_vpc.main.id
  cidr_block     = var.public_subnets[count.index]
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name      = "${var.environment}-public-${count.index}"
    Environment = var.environment
  }
}
```





```
resource "aws_subnet" "private" {
  count          = length(var.private_subnets)
  vpc_id         = aws_vpc.main.id
  cidr_block     = var.private_subnets[count.index]
  availability_zone = data.aws_availability_zones.available.names[count.index]
  tags = {
    Name      = "${var.environment}-private-${count.index}"
    Environment = var.environment
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
  tags = {
    Name      = "${var.environment}-public-rt"
    Environment = var.environment
  }
}

resource "aws_route_table_association" "public" {
  count          = length(var.public_subnets)
  subnet_id     = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}
```





```
resource "aws_eip" "nat" {
  count = length(var.public_subnets)
  vpc    = true
  tags = {
    Name      = "${var.environment}-nat-eip-${count.index}"
    Environment = var.environment
  }
}

resource "aws_nat_gateway" "main" {
  count          = length(var.public_subnets)
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
  tags = {
    Name      = "${var.environment}-nat-gw-${count.index}"
    Environment = var.environment
  }
}

resource "aws_route_table" "private" {
  count = length(var.private_subnets)
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[count.index].id
  }
  tags = {
    Name      = "${var.environment}-private-rt-${count.index}"
    Environment = var.environment
  }
}
```





```
resource "aws_route_table_association" "private" {
  count          = length(var.private_subnets)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private[count.index].id
}

data "aws_availability_zones" "available" {
  state = "available"
}
```

modules/networking/variables.tf

```
variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
}

variable "public_subnets" {
  description = "List of public subnet CIDR blocks"
  type        = list(string)
}

variable "private_subnets" {
  description = "List of private subnet CIDR blocks"
  type        = list(string)
}

variable "environment" {
  description = "Environment name"
  type        = string
}
```

modules/networking/outputs.tf





```
output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "IDs of public subnets"
  value       = aws_subnet.public[*].id
}

output "private_subnet_ids" {
  description = "IDs of private subnets"
  value       = aws_subnet.private[*].id
}

output "nat_gateway_ips" {
  description = "IP addresses of NAT gateways"
  value       = aws_eip.nat[*].public_ip
}
```

3. Compute Module

modules/compute/main.tf





```
resource "aws_security_group" "web" {
  name           = "${var.environment}-web-sg"
  description    = "Security group for web servers"
  vpc_id        = var.vpc_id

  ingress {
    from_port     = 80
    to_port       = 80
    protocol      = "tcp"
    cidr_blocks   = ["0.0.0.0/0"]
  }

  ingress {
    from_port     = 22
    to_port       = 22
    protocol      = "tcp"
    cidr_blocks   = ["0.0.0.0/0"]
  }

  egress {
    from_port     = 0
    to_port       = 0
    protocol      = "-1"
    cidr_blocks   = ["0.0.0.0/0"]
  }

  tags = {
    Name           = "${var.environment}-web-sg"
    Environment    = var.environment
  }
}
```





```
resource "aws_launch_configuration" "web" {
  name_prefix    = "${var.environment}-web-"
  image_id       = data.aws_ami.ubuntu.id
  instance_type  = var.instance_type
  key_name       = var.key_name
  security_groups = [aws_security_group.web.id]

  user_data = <<-EOF
    #!/bin/bash
    apt-get update
    apt-get install -y apache2
    echo "<h1>Hello from ${var.environment} environment</h1>" > /var/www/html/index.html
    systemctl start apache2
    systemctl enable apache2
  EOF

  lifecycle {
    create_before_destroy = true
  }
}
```





```
resource "aws_autoscaling_group" "web" {
  name                = "${var.environment}-web-asg"
  launch_configuration = aws_launch_configuration.web.name
  min_size            = 2
  max_size            = 4
  desired_capacity     = 2
  vpc_zone_identifier = var.private_subnet_ids

  tag {
    key          = "Name"
    value        = "${var.environment}-web-instance"
    propagate_at_launch = true
  }
}

resource "aws_lb" "web" {
  name                = "${var.environment}-web-alb"
  internal            = false
  load_balancer_type = "application"
  security_groups     = [aws_security_group.web.id]
  subnets            = var.public_subnet_ids

  tags = {
    Name          = "${var.environment}-web-alb"
    Environment = var.environment
  }
}
```





```
resource "aws_lb_target_group" "web" {
  name      = "${var.environment}-web-tg"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = var.vpc_id

  health_check {
    path            = "/"
    interval        = 30
    timeout         = 5
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}

resource "aws_autoscaling_attachment" "web" {
  autoscaling_group_name = aws_autoscaling_group.web.id
  alb_target_group_arn   = aws_lb_target_group.web.arn
}

resource "aws_lb_listener" "web" {
  load_balancer_arn = aws_lb.web.arn
  port              = "80"
  protocol           = "HTTP"

  default_action {
    type            = "forward"
    target_group_arn = aws_lb_target_group.web.arn
  }
}
```





```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

modules/compute/variables.tf





```
variable "vpc_id" {
  description = "ID of the VPC"
  type        = string
}

variable "public_subnet_ids" {
  description = "IDs of public subnets"
  type        = list(string)
}

variable "private_subnet_ids" {
  description = "IDs of private subnets"
  type        = list(string)
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "key_name" {
  description = "Name of the EC2 key pair"
  type        = string
}

variable "environment" {
  description = "Environment name"
  type        = string
}
```

modules/compute/outputs.tf





```
output "web_server_public_ips" {
  description = "Public IP addresses of web servers"
  value       = aws_instance.web[*].public_ip
}

output "alb_dns_name" {
  description = "DNS name of the Application Load Balancer"
  value       = aws_lb.web.dns_name
}
```

4. Database Module

modules/database/main.tf





```
resource "aws_security_group" "db" {
  name          = "${var.environment}-db-sg"
  description   = "Security group for database"
  vpc_id        = var.vpc_id

  ingress {
    from_port    = 3306
    to_port      = 3306
    protocol     = "tcp"
    cidr_blocks  = ["10.0.0.0/16"]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  tags = {
    Name          = "${var.environment}-db-sg"
    Environment   = var.environment
  }
}
```





```
resource "aws_db_subnet_group" "main" {
  name          = "${var.environment}-db-subnet-group"
  subnet_ids    = var.private_subnet_ids

  tags = {
    Name          = "${var.environment}-db-subnet-group"
    Environment    = var.environment
  }
}

resource "aws_db_instance" "main" {
  identifier          = "${var.environment}-db"
  allocated_storage   = 20
  storage_type        = "gp2"
  engine              = "mysql"
  engine_version      = "5.7"
  instance_class      = "db.t2.micro"
  name                = var.db_name
  username            = var.db_username
  password            = var.db_password
  parameter_group_name = "default.mysql5.7"
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]
  skip_final_snapshot = true
  publicly_accessible = false

  tags = {
    Name          = "${var.environment}-db-instance"
    Environment    = var.environment
  }
}
```

modules/database/variables.tf





```
variable "vpc_id" {
  description = "ID of the VPC"
  type        = string
}

variable "private_subnet_ids" {
  description = "IDs of private subnets"
  type        = list(string)
}

variable "db_name" {
  description = "Database name"
  type        = string
}

variable "db_username" {
  description = "Database username"
  type        = string
}

variable "db_password" {
  description = "Database password"
  type        = string
  sensitive   = true
}

variable "environment" {
  description = "Environment name"
  type        = string
}
```

modules/database/outputs.tf

```
output "database_endpoint" {
  description = "Endpoint of the RDS database"
  value       = aws_db_instance.main.endpoint
}

output "database_name" {
  description = "Name of the database"
  value       = aws_db_instance.main.name
}
```

5. Example Usage





examples/dev/main.tf

```
module "infrastructure" {
  source      = "../.."
  environment = "dev"
  key_name    = "dev-key"

  db_name      = "devdb"
  db_username  = "admin"
  db_password  = "devpassword123"
}
```

examples/dev/terraform.tfvars

aws_region = "us-east-1"

examples/prod/main.tf

```
module "infrastructure" {
  source      = "../.."
  environment = "prod"
  key_name    = "prod-key"

  instance_type = "t2.medium"

  db_name      = "proddb"
  db_username  = "admin"
  db_password  = "prodpassword456"
}
```

examples/prod/terraform.tfvars

```
aws_region = "us-west-2"
vpc_cidr   = "10.1.0.0/16"
public_subnets = ["10.1.1.0/24", "10.1.2.0/24"]
private_subnets = ["10.1.3.0/24", "10.1.4.0/24"]
```

Key Module Concepts Demonstrated





1. **Module Composition:** The root module composes multiple child modules (networking, compute, database).
2. **Input Variables:** Each module defines its required inputs in variables.tf.
3. **Output Values:** Modules expose outputs in outputs.tf that can be used by other modules.
4. **Module Reusability:** The same modules can be used for different environments (dev, prod).
5. **Module Dependencies:** Modules can depend on outputs from other modules (e.g., compute module needs VPC ID from networking module).
6. **Module Versioning:** While not shown here, you can version modules using Git tags or Terraform Registry.
7. **Local Modules:** These are modules stored in the same codebase (using source = "./modules/name").
8. **Remote Modules:** You can also use modules from Terraform Registry or other repositories.
9. **Module Organization:** Logical grouping of related resources (networking, compute, database).
10. **Module Testing:** The examples directory provides example configurations for testing.

Advanced Module Features

1. **Dynamic Blocks:** Within modules, you can use dynamic blocks to create multiple similar configurations.
2. **Count and For Each:** Modules themselves can be called multiple times using count or for_each.
3. **Module Aliases:** When calling the same module multiple times, you can use aliases to distinguish them.





4. Conditional Resources: Modules can include resources that are conditionally created based on input variables.

5. Data Sources: Modules can use data sources to fetch information from the provider.

6. Local Values: Modules can define local values for complex expressions used in multiple places.

Best Practices for Terraform Modules

1. Single Responsibility: Each module should have a single, well-defined purpose.

2. Documentation: Include README files explaining module purpose, inputs, and outputs.

3. Versioning: Version your modules when publishing them for reuse.

4. Testing: Include example configurations and consider automated testing.

5. Input Validation: Use variable validation to ensure proper inputs.

6. Sensitive Variables: Mark sensitive variables appropriately.

7. Output Documentation: Clearly document all outputs.

8. Standard Structure: Follow a consistent structure across modules.

9. Minimize Provider Configuration: Avoid provider configurations in modules.

10. Tagging: Consistently tag resources for better management.

